

# Agent 从“能跑”到“可治理”

## InfoQ 2026 年 7 月两篇文章的深度研究与工程化推演

Runtime × Harness × Memory × Governance × Organization

### 研究对象

《Agent 狂欢热潮下的冷思考：为什么规模化落地总是陷入僵局？》 | InfoQ | 2026-07-02

《Agent 上岗之后，企业如何治理硅基团队？》 | InfoQ | 2026-07-03

研究版 · 2026-08

## 0. 执行摘要：两篇文章合起来到底在讲什么？

### 一句话结论

7月2日的文章解决“Agent 怎么稳定、安全、低成本地活下来”，核心是 Agent Runtime；7月3日的文章解决“Agent 活下来以后，如何成为组织生产力的一部分”，核心是 Harness、Memory 与 Human-Agent 治理。两篇拼起来，指向的是一套正在成型的 Enterprise Agent Operating Environment。

这两篇文章看似一篇偏基础设施、一篇偏组织治理，但它们的底层逻辑高度一致：当 Agent 从“回答问题的软件能力”变成“能够持续执行任务、调用工具、改变外部世界的生产力单元”后，工程对象就不再是一次模型请求，而是一个具有生命周期、状态、权限、成本、责任和学习轨迹的任务系统。

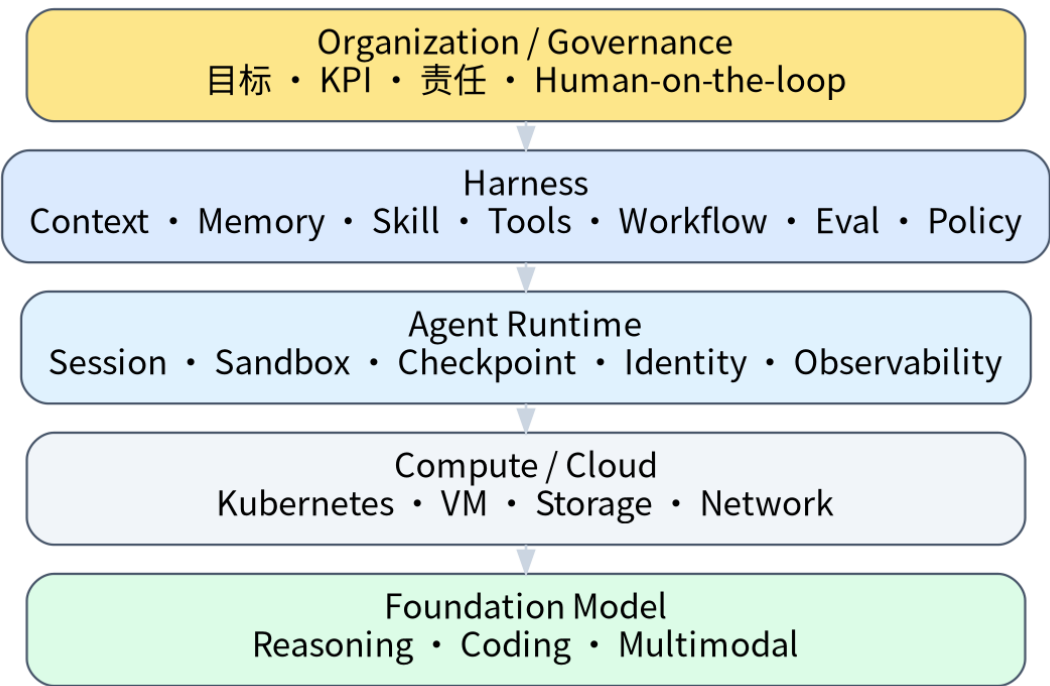


图 0-1：两篇文章合并后得到的企业 Agent 技术栈

| 层次               | 核心问题                    | 两篇文章覆盖重点              |
|------------------|-------------------------|-----------------------|
| Foundation Model | 模型能否理解、推理、生成与规划？        | 两篇均视为基础能力，但不是规模化落地的全部 |
| Agent Runtime    | Agent 在哪里运行？如何隔离、恢复、扩容？ | 7月2日主线                |
| Harness          | 模型如何被工具、记忆、工作流、评测、策略包裹？ | 7月3日主线                |

|                           |                   |                  |
|---------------------------|-------------------|------------------|
| Organization / Governance | 谁负责目标、审批、责任与组织协作？ | 7 月 3 日进一步上升到组织层 |
|---------------------------|-------------------|------------------|

本报告采用三层证据标记：①“原文”表示 InfoQ 报道直接提出；②“验证”表示可从 IDC、腾讯 CubeSandbox、AWS、Anthropic、华为云等一手资料交叉确认；③“推演”表示基于上述事实进一步抽象出的工程模型。

### 0.1 最值得记住的 8 个判断

1. Agent 的基本工作单元正在从 Request 转向 Task；因此 Session、Checkpoint、Pause/Resume、Trajectory 变成一等能力。
2. Agent 的主要风险从 Output Risk 转向 Action Risk；因此身份、凭证、工具网关、审批、审计和回滚比传统 Chatbot 更重要。
3. Agent Runtime 更像新的工作负载语义层，而不是 Kubernetes 的简单替代品；底层仍可建立在 VM、Kubernetes、Serverless 等资源层之上。
4. Sandbox 不是单纯“安全容器”，而是 Agent 的可复制、可回滚、可分叉执行世界；这对长任务、RL rollout 与故障恢复尤其关键。
5. Memory 不等于 Vector DB。真正的企业 Memory 还要处理来源、时间、冲突、版本、归属、写权限、遗忘与演进。
6. Harness 的本质不是“加更多 Prompt 约束”，而是把模型放进真实生产关系：工具、上下文、记忆、策略、评测、角色与责任。
7. Human in the Loop 负责具体异常点；Human on the Loop 负责系统级规则、阈值、监控与升级机制。
8. 企业最终优化目标不是 Token 单价，而是 Cost per Successful Outcome，以及组织级吞吐和可靠性。

# 1. 第一篇：为什么 Agent 规模化落地首先撞上 Runtime ？

## 1.1 文章的核心命题：难点正在从模型迁移到运行环境

【原文】InfoQ 将生产级 Agent 与传统 AI 应用区分开：传统应用更接近“请求—模型—返回”的单次调用，而生产 Agent 会持续访问数据、调用工具、操作文件、与其他 Agent 协作，并在较长时间内推进任务。[1] 这使其工程形态更接近持续运行的软件系统。

【验证】IDC 2026 年 1 月的预测给出了规模背景：全球活跃 Agent 从 2025 年约 2860 万增长到 2030 年 22.16 亿，年执行任务量从约 440 亿次增长到 415 万亿次。[3] 即便把预测本身视为情景估计，而不是确定事实，它所揭示的趋势仍然明确：任务数量、交互深度和推理链路增长会把系统工程问题放大。



图 1-1：从 Request-centric 到 Task-centric 的工作负载迁移

### Know-why

为什么 Runtime 会在 2026 年突然重要？因为一次请求失败，只需要重试；但一个已经执行 40 分钟、创建文件、修改代码、调用外部系统的任务失败，意味着状态、成本和真实副作用都要处理。任务生命周期一旦成为主角，Runtime 就从“部署细节”变成核心产品能力。

## 1.2 三个规模化难题：弹性、运维、治理

【原文】第一篇把复杂问题抽象成三个维度：弹性、运维、治理。[1] 这是全文最有复用价值的抽象，因为它能直接转化成 Agent Infra 的非功能需求。

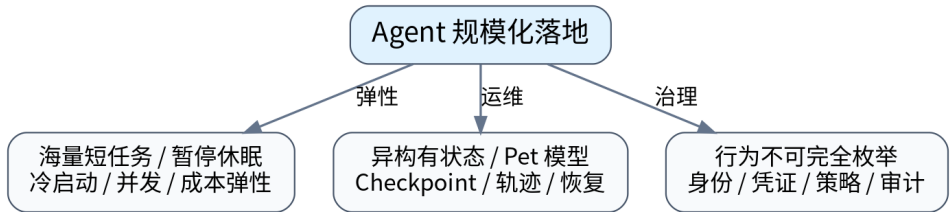


图 1-2：InfoQ 第一篇抽象出的三类基础设施挑战

| 维度 | 传统服务的典型假设 | Agent 的新特性 | 必须补齐的能力 |
|----|-----------|------------|---------|
|----|-----------|------------|---------|

|    |                    |                     |                                                        |
|----|--------------------|---------------------|--------------------------------------------------------|
| 弹性 | 服务副本相对稳态；按 QPS 扩缩容 | 海量任务实例、突发并发、频繁暂停/恢复 | 快速冷启动、资源池、AutoPause/Resume、任务级调度                       |
| 运维 | 同质、无状态或状态外置        | 异构、有状态、执行轨迹各不相同     | Checkpoint、Snapshot、Trajectory、Session 级可观测            |
| 治理 | API 行为相对可枚举        | 模型决定下一步动作，行为路径不固定   | Identity、Credential、Policy、Tool Gateway、Approval、Audit |

### 1.3 为什么 Kubernetes / Serverless 会“拧巴”？

【原文】InfoQ 的论点是：现有云原生技术能解决部分“计算与部署”问题，但不是围绕 Agent 的任务语义设计。Serverless Container 往往围绕请求连接管理实例，而 Agent 的真正生命周期绑定在任务；Kubernetes 更擅长相对稳态、服务型工作负载，而 Agent 可能出现大量短生命周期实例、暂停休眠和个体化状态。[1]

【推演】这里最容易形成的误读是“Agent Runtime 会替代 Kubernetes”。更准确的理解是：Agent Runtime 提供更高一层的任务语义和状态语义，而 Kubernetes/VM/Cloud 仍然可以作为底层资源层。类似 JVM 没有替代操作系统，而是把 Java 应用所需的执行语义封装出来。

#### 更准确的层次关系

Cloud / Kubernetes / VM 负责机器、节点、网络 and 基础资源；Agent Runtime 负责 Session、Sandbox、Checkpoint、Identity、Task Lifecycle、Trajectory 等 Agent 原生语义。Runtime 是 workload abstraction layer，而不是对底层资源系统的否定。

### 1.4 腾讯云 Agent Runtime 的四层结构

【原文】InfoQ 将腾讯云 Agent Runtime 描述为“接入、运行、治理、智能”四层：接入解决多框架多协议兼容；运行提供执行引擎、安全沙箱、会话快照与持久化；治理覆盖工具网关、身份凭证、策略管控与运行观测；智能层则承载记忆、评估、Skill 与知识库。[1]

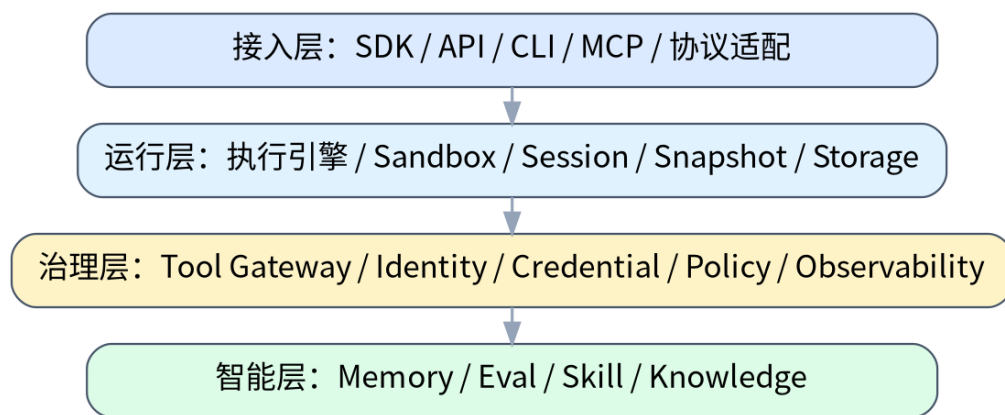


图 1-3：Agent Runtime 四层能力模型

【推演】这个四层模型的重要性在于，它把 Agent Runtime 从“一个容器管理器”提升为生命周期平台：接入层降低框架锁定；运行层解决可靠执行；治理层解决企业是否敢放权；智能层解决是否能在任务历史中持续改进。

## 2. Sandbox 与 State：真正的核心不是容器，而是“可恢复世界”

### 2.1 CubeSandbox 为什么是这篇文章里最值得研究的工程细节

【验证】CubeSandbox 官方 GitHub 显示，它基于 RustVMM 与 KVM，目标是提供硬件级隔离、快速冷启动与高密度运行；官方给出的平均端到端冷启动指标低于 60ms、单实例额外内存开销低于 5MB。[4] 这些属于项目官方性能主张，适合用于理解设计目标，而不应当等同于所有生产负载下的通用基准。

【验证】CubeSandbox v0.3.0 引入 CubeCoW，支持快照、克隆与回滚，并使用增量内存快照等机制降低重复快照成本。[5] 这说明 Sandbox 的演进重点已经从“启动一个隔离环境”走向“管理环境的时间状态”。

### 2.2 Snapshot 的意义：不仅是故障恢复

传统服务的快照常用于备份或恢复；对 Agent 来说，快照还可以成为搜索与探索的基础原语。某个执行状态可以被保存，然后从同一 checkpoint 分叉多个候选轨迹，分别尝试不同工具、代码修改或计划，再由评测器选择结果。

| 用途                 | 为什么 Snapshot 有价值                 |
|--------------------|----------------------------------|
| 故障恢复               | 长任务失败时不必从零开始，减少 token、工具与人力重试成本  |
| Best-of-N / Search | 从同一世界状态分叉多条 trajectory，比较不同计划与结果 |
| Agentic RL Rollout | 大量并行环境需要快速复制同一初始状态，再让策略独立探索      |
| 调试与复现              | 将“出错前的世界”保存下来，便于重放 Agent 轨迹和定位根因 |

### 2.3 为什么 Agent Checkpoint 不能只保存 messages[]

【推演】如果 Agent 已经创建 Git 分支、修改文件、启动进程、访问外部 API、提交工单或触发部署，那么只恢复对话消息并不能恢复“真实世界”。因此生产级 checkpoint 必须至少区分模型上下文状态、任务状态、环境状态、外部副作用和恢复语义。

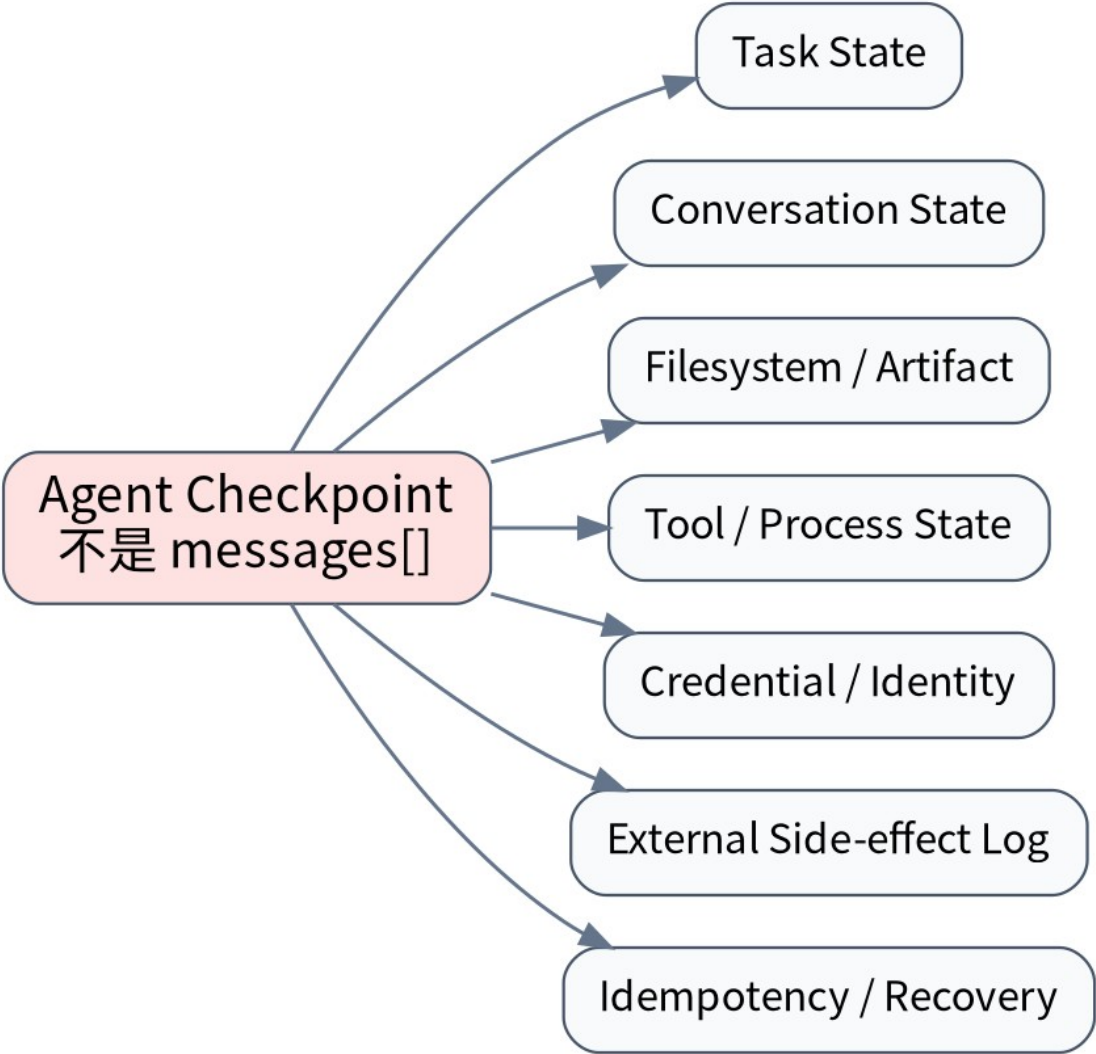


图 2-1：生产级 Agent Checkpoint 的状态构成

工程原则

Checkpoint 不是“保存聊天记录”；它更接近事务系统里的恢复点。一个可靠设计必须回答：恢复后哪些动作已经发生？哪些动作可以安全重试？哪些外部系统无法自动回滚？是否具备 idempotency key、action log 和人工补偿路径？

2.4 一个可复用的 State Manager 接口

如果自己搭建 Agent Runtime MVP，State Manager 可以先做成简单但语义清晰的接口，而不是一开始追求完整分布式快照：

| 接口 | 语义 |
|----|----|
|----|----|

|                             |                                 |
|-----------------------------|---------------------------------|
| create_checkpoint(task_id)  | 保存逻辑任务进度 + 环境引用 + action log 游标 |
| pause(task_id)              | 停止继续执行，但保留可恢复状态                 |
| resume(task_id, checkpoint) | 从明确恢复点继续，重新验证凭证与资源              |
| record_side_effect(action)  | 记录不可逆或需要补偿的外部动作                 |
| fork(checkpoint, n)         | 为搜索 / 评测 / rollout 创建 N 个独立后继环境 |

### 3. 从 Output Risk 到 Action Risk：为什么治理成为 Runtime 的一等能力

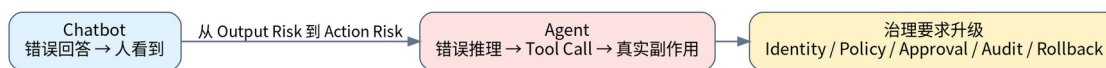


图 3-1：Agent 把“信息错误”升级为“行动错误”

Chatbot 的错误通常停留在输出层；Agent 的错误可能继续转化为工具调用，并最终修改文件、数据库、工单、代码仓库、业务配置或其他外部状态。这意味着企业安全问题不再只是“模型有没有幻觉”，而是“错误是否获得了执行权限”。

#### 3.1 一个企业级 Agent 权限链必须回答 7 个问题

9. Who：当前执行主体是谁？是 Agent 自身身份，还是代表某个用户？
10. For whom：它正在代表谁行动？用户授权是否仍然有效？
11. What：允许调用哪些工具、API、数据集和执行环境？
12. How much：参数、额度、频率、资源和数据范围能到什么程度？
13. When to approve：哪些不可逆、高影响或越界动作必须人工批准？
14. What happened：能否用 trace / action log 还原完整执行轨迹？
15. How to recover：失败后是重试、补偿、回滚还是人工处理？

【验证】AWS AgentCore 目前同样把 Runtime、Memory、Gateway、Identity、Observability、Evaluations 等拆为独立模块；其官方文档强调 Runtime 的 session isolation、scaling 和身份门控，Gateway 提供集中式工具访问，Identity 处理认证授权，Observability/Evaluations 用于持续观测和评估。[6][7] 这与 InfoQ 第一篇的治理层抽象高度同构。

#### 3.2 Credential：不要把密交 Sandbox

【推演】Agent 的执行环境天然更高风险：模型可能读取环境变量、执行未知代码或访问不可信依赖。因此凭证管理应尽量从“把 secret 注入容器”升级为“凭证代理 / 令牌交换 / 短期授权”。Agent 只看到完成当前动作所需的最小能力，而不是长期凭证本体。

##### 最小权限不是静态 RBAC 就够了

Agent 的能力边界需要同时考虑“身份 + 工具 + 参数 + 时间 + 上下文 + 风险等级”。一个允许调用 issue.update 的 Agent，不代表它在所有项目、所有字段、所有时间都应被允许执行。

## 4. 第二篇：Agent “上岗”之后，为什么问题从工程走向组织？

### 4.1 核心命题：企业需要的不是更多 Agent，而是一套 Harness 系统

【原文】7月3日的 InfoQ 报道把问题定义为：当 Agent 进入研发、运营、数据分析和业务交付等核心流程，企业需要建立可控、可信、可追责、可持续进化的 Harness 系统。[2] 这个定义很重要，因为它把 Harness 从“模型调用框架”提升成“生产关系框架”。

【验证】华为云 2026 年产品文档直接给出“Agent = Model + Harness”的定义，并把 Harness 描述为决定模型能看到什么、能做什么、何时停止、出错时如何处理的基础设施层。[8] Anthropic 2026 年 3 月的工程文章也明确把 harness design 视为长程 Agent 性能的重要组成部分。[9]

#### Harness 的合理定义

Harness = Context + Memory + Skills + Tools + Workflow + Policy + Observability + Evaluation + Human Intervention + Multi-agent Coordination。它的目标不是“把模型绑死”，而是给概率性模型提供可复用的执行环境、反馈结构和责任边界。

### 4.2 Model 定原始能力，Harness 决定能力如何被组织

| 组件                   | 解决的问题      | 失败时的典型症状           |
|----------------------|------------|--------------------|
| Context              | 本步推理看什么    | 遗漏约束、重复搜索、上下文污染    |
| Tools                | 能对世界做什么    | 工具选错、参数错误、权限越界     |
| Memory               | 跨任务保留什么    | 重复犯错、历史冲突、经验无法复用   |
| Workflow / Subagent  | 如何拆任务与协同   | 串行瓶颈、角色混乱、缺少反证     |
| Policy / Approval    | 哪里必须收敛不确定性 | 高风险动作直接执行          |
| Observability / Eval | 怎么知道好不好    | 成功率不可量化、成本与失败原因不可见 |

## 5. Memory：第二篇最值得沉淀的工程思想

### 5.1 Memory 不等于向量数据库

【原文】周祥把记忆系统拆成“记忆蒸馏、记忆计算、记忆堆叠”三个层次。[2] 这个分法的价值在于：它把企业 Memory 从单纯“检索存储”变成了知识生命周期系统。

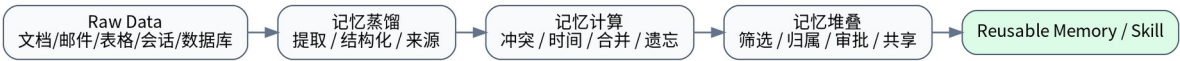


图 5-1：从原始企业数据到可治理组织记忆

## 5.2 记忆蒸馏：把 Legacy Data 变成 AI Ready Data

企业经验分散在数据库、文档、PDF、表格、邮件、会话、SOP、代码、工单和人的隐性知识里。它们并不天然具有统一结构，也不天然适合检索。记忆蒸馏首先需要保留来源与结构，再进行提取、归一化和语义化，而不是把所有内容直接切 chunk。

| 字段                   | 为什么必要            |
|----------------------|------------------|
| content              | 记忆的主体信息          |
| source / provenance  | 知道它来自哪里，能否回查     |
| timestamp / validity | 判断信息在什么时间范围内有效   |
| scope / project      | 控制在什么业务范围内生效     |
| actor / owner        | 明确责任归属与写入主体      |
| confidence / quality | 区分已验证事实、推断和低质量经验 |
| version              | 允许演进、回滚、对比       |
| permission           | 控制谁能读、谁能写、谁能共享   |

## 5.3 记忆计算：真正难的是“冲突与时间”

【原文】第二篇特别强调冲突检测、遗忘、合并、演进、时间一致性、跨会话综合、抗幻觉和语义模糊。[2] 这揭示了企业 Memory 的本质：知识不是一个永远正确的静态集合，而是一组随时间演进、可能互相矛盾的事实、规则和决策。

### 从 RAG 到 Memory Compute

RAG 的典型问题是“给定 query，找最相关 chunks”；Memory Compute 还要回答“这些 chunks 是否彼此冲突？哪个更新？适用范围是否相同？有没有例外？旧规则是否应该被废弃？”

## 5.4 记忆堆叠：最大的风险是“把错误制度化”

当一次错误输出只存在于本次上下文里，影响范围有限；但如果它被写入长期记忆、被后续 Agent 反复检索、再被提炼成 Skill 并分发给组织，错误会获得复制能力。这就是组织级 Memory Poisoning 的危险。

【原文】第二篇提出 Workspace、Actor、Project 的组织框架，并强调记忆写入必须落到明确的主 Project 和 Actor 下。[2] 这实际上说明：长期 Memory write 更接近“代码提交”而不是“缓存写入”。

| 普通缓存写入    | 组织记忆写入              |
|-----------|---------------------|
| 目标：提高性能   | 目标：改变未来决策依据         |
| 一般不需要人工责任 | 需要来源、作者/Actor 与业务归属 |
| 错误通常可被淘汰  | 错误可能被长期复用和放大        |
| 很少需要评审    | 高价值共享记忆应具备验证、审批和回滚  |

## 6. Agent FinOps：为什么“Token 单价”不是正确成本指标

### 6.1 从 Token Price 转向 Cost per Successful Outcome

【原文】第二篇引用一个简化公式：Token 成本约等于 Token 单价除以任务成功率。[2] 它并不是完整财务模型，但抓住了关键方向：便宜但失败率高的 Agent，可能比单次调用更贵但成功率高的 Agent 成本更高。

【推演】更完整的期望成功成本可以拆成：模型 token + 工具调用 + 计算/Sandbox + 重试 + 人工介入 + 失败恢复。真正需要优化的是“成功交付一个业务结果的期望成本”，而不是某个模型的每百万 Token 标价。

| 推荐指标                         | 解释                    |
|------------------------------|-----------------------|
| Cost / Successful Task       | 最接近业务结果的总成本           |
| Tokens / Successful Task     | 避免只看单次调用而忽略 retry     |
| Retries / Task               | 揭示上下文、工具、规划或模型不稳定     |
| Tool Calls / Task            | 发现冗余搜索与工具选择问题         |
| Human Interventions / Task   | 衡量自动化程度和异常质量          |
| Latency / Successful Task    | 从用户和流程视角衡量交付速度        |
| Rollback / Compensation Rate | 衡量 Action Risk 实际发生频率 |

#### Know-how

给每次 Agent Task 分配稳定 task\_id，把 model span、tool span、sandbox span、approval span、memory read/write、外部副作用和最终 eval score 串到同一条 trace 上。否则“成本”和“成功率”无法在任务级被正确关联。

## 7. Human-Agent 治理：“人批每一步”到“人系界”

### 7.1 Human in the Loop 与 Human on the Loop

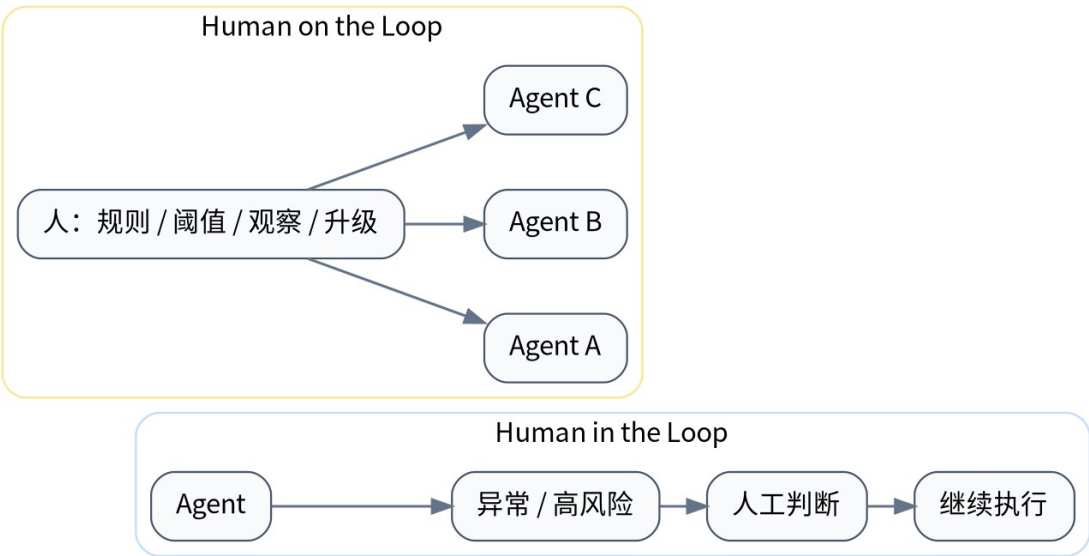


图 7-1：两种不同的人类参与机制

【原文】第二篇把 Human on the Loop 描述为人在系统上方进行治理：制定规则、观察状态和控制边界；Human in the Loop 则是在异常、分歧或高风险节点进入具体流程。[2] 两者并不是二选一，而是应该组合使用。

| 任务类型                 | 建议治理方式                                  |
|----------------------|-----------------------------------------|
| 研究、草案、可逆原型           | 以 Human-on-the-loop 为主：宽松目标 + 观测 + 结果评审 |
| 修改代码但尚未合并            | Agent 自主执行 + 自动测试 + PR 审核               |
| 生产部署、资金/权限变更、不可逆外部动作 | Human-in-the-loop 强审批 + 最小权限 + 完整审计     |
| 高频低风险后台任务            | 策略阈值 + 抽样审计 + 异常升级                      |

### 7.2 Multi-Agent 的价值不是“多几个模型一起聊天”

【原文】华为 ACE Harness 案例包含 Worker、蓝军/反向审视 Agent、裁判 Agent，以及必要时的人工介入。[2] 报道中的效率数据来自现场分享，缺乏公开独立 benchmark，因此更适合作为实践线索而不是严格因果结论。

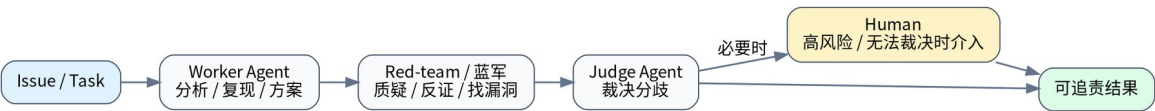


图 7-2：更可靠的 Multi-Agent 结构来自角色分离，而非简单并行

结构性 know-why

Generator、Critic、Judge、Executor、Approver 最好不是同一个角色，因为它们的优化目标不同。角色分离相当于把软件工程里的开发、Review、CI、审批机制迁移到 Agent 系统中。

8. 为什么“每人一个 Agent”并不会自动变成组织生产力

8.1 Context Fragmentation：50 个人可能得到 50 个不同的 Claude Code

【原文】第二篇指出，即使使用同一款工具，不同人的 Prompt、表达方式和上下文理解也会让团队形成完全不同的实际工作方式；嘉宾甚至用“50 个人可能变成 50 种工具”来描述这种现象。[2]

【推演】传统协作依赖共享文档、代码、会议和工单，而 Human-Agent 工作大量发生在个人上下文窗口里。如果团队没有共享术语、架构规则、Skill、Memory、Eval 与交付标准，AI 反而可能放大“个人局部最优”，导致 Context Fragmentation。

| 个人智能资产      | 需要外化成的组织资产                                    |
|-------------|-----------------------------------------------|
| 某人的好 Prompt | 可测试的 Skill / Procedure                        |
| 某次成功调试路径    | Runbook / Failure Pattern / Eval Case         |
| 某人的业务背景知识   | 带来源与权限的 Shared Memory                         |
| 某人的工具习惯     | 统一 Tool Schema / Gateway / Usage Policy       |
| 某人的质量判断     | Rubric / Test / Judge / Human Review Standard |

8.2 Harness 不是越多越好：Minimum Sufficient Control

【原文】圆桌中也出现了重要反方观点：过度设定 Harness 可能限制模型能力，在探索型任务上给粗粒度目标、让模型先生成再由人修剪，有时效果更好。[2] Anthropic 2026 年关于 Managed Agents 的文章也强调，Harness 会编码对模型能力的假设，而这些假设会随着模型变强而过时。[10]

【推演】因此，正确目标不是最大化控制，而是 Minimum Sufficient Control：根据任务风险、可逆性、不确定性和业务影响动态决定 Harness 强度。

| 场景              | Harness 强度 | 原因                     |
|-----------------|------------|------------------------|
| 研究 / Brainstorm | 低-中        | 保留探索空间，重点约束来源、结果格式和成本  |
| 代码实现 / 测试       | 中          | 允许自主迭代，但用测试、Git、Review |

|              |     |                    |
|--------------|-----|--------------------|
|              |     | 形成反馈闭环             |
| 客户沟通 / 业务操作  | 中-高 | 加入身份、内容策略和关键动作审批   |
| 生产变更 / 高风险动作 | 高   | 强权限、强审计、强审批、可回滚/补偿 |

## 9. 把两篇文章合并：一套 Enterprise Agent Operating Environment

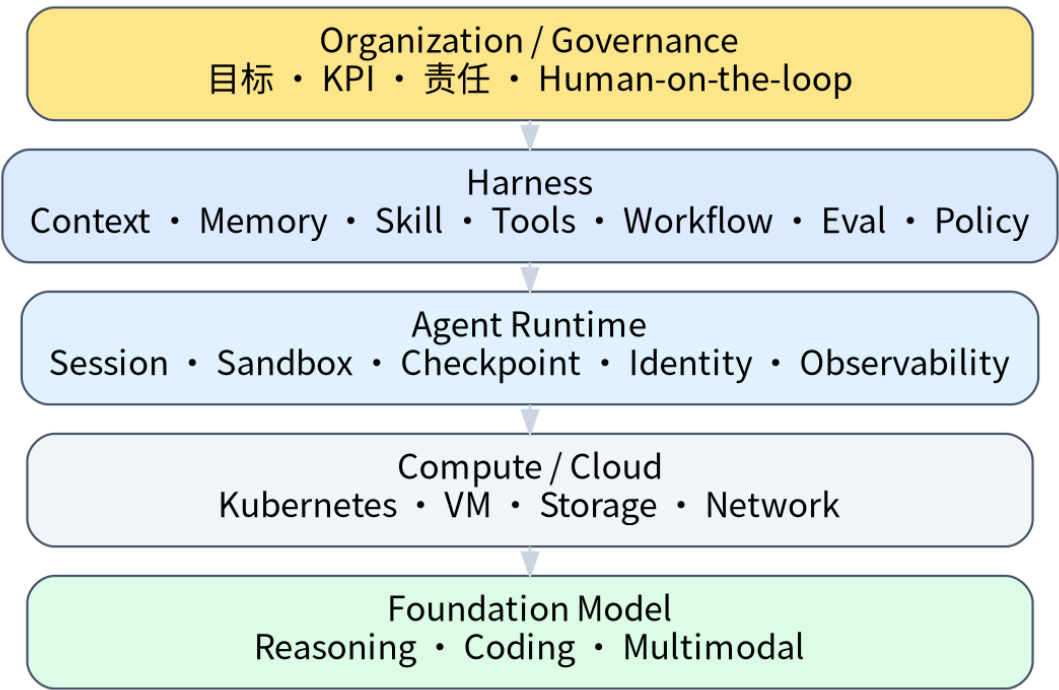


图 9-1：Enterprise Agent Operating Environment 的五层模型

这个五层结构提供了一个重要的排错顺序：当 Agent 表现差时，不要立即归因于模型。先判断问题属于模型能力、上下文/记忆、工具/Harness、Runtime 可靠性，还是组织目标与流程设计。很多“模型不行”的问题实际上发生在其他层。

| 现象            | 优先检查层                                     |
|---------------|-------------------------------------------|
| 经常忘记前序工作、重复做事 | Context / Memory / State                  |
| 经常调用错工具或参数    | Tool Design / Harness / Eval              |
| 长任务中断后无法恢复    | Runtime / Checkpoint / Artifact State     |
| 结果正确但成本爆炸     | Harness / Memory / Tool Strategy / FinOps |
| 低风险任务也处处人工审批  | Governance 过强，Harness 过度约束                |

|              |                                             |
|--------------|---------------------------------------------|
| 团队每个人效果差异极大  | Organization Context / Shared Skill / Eval  |
| 错误动作已经影响真实系统 | Identity / Policy / Approval / Action Audit |

9.1 五个闭环：判断一个平台是否“生产级”的快速框架

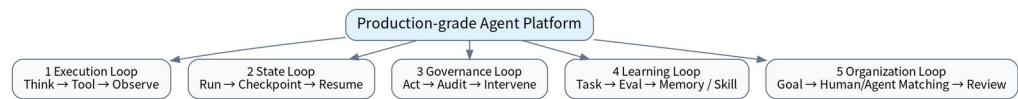


图 9-2：生产级 Agent 平台的五个闭环

很多 Demo 只有第一个 Execution Loop：模型调用工具，然后把结果喂回模型。生产系统必须逐步补齐 State、Governance、Learning 和 Organization 四个闭环。缺少任一闭环，都可能在规模化后暴露出系统性问题。

10. 工程落地：把文章转化为 Agent Runtime MVP 蓝图

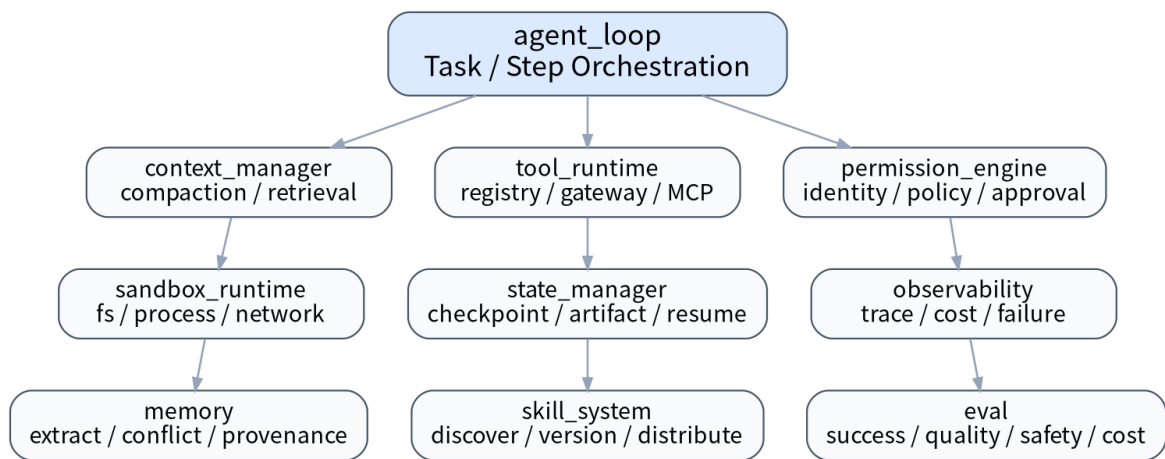


图 10-1：从简单 Agent Loop 演进为生产 Runtime 的模块图

10.1 推荐的演进顺序

- 16. Agent Loop：先确保模型—工具—结果回传闭环正确，建立稳定 task\_id。
- 17. Tool Runtime：工具注册、Schema、错误模型、超时、幂等与统一分发。
- 18. Permission Engine：硬拒绝、规则匹配、用户审批、身份/凭证边界。
- 19. Sandbox Runtime：文件、进程、网络隔离；危险能力默认不暴露。
- 20. Observability：每个模型/工具/审批/环境操作生成统一 trace。
- 21. State Manager：checkpoint、pause/resume、artifact tracking、external action log。
- 22. Context Manager：上下文选择、压缩、检索、会话交接。
- 23. Memory：先做带 provenance 的长期事实与经验，再做冲突、版本和治理。

- 24. Skill System：把高价值重复成功路径外化、评测、版本化，再分发。
- 25. Eval / AgentOps：按成功任务衡量质量、成本、安全与回归。

10.2 每模的最小接口

| 模块                | MVP 最小职责                                          | 不要一开始做什么                  |
|-------------------|---------------------------------------------------|---------------------------|
| tool_runtime      | 统一 schema、dispatch、timeout、typed result           | 不要为每个工具写完全不同的返回协议         |
| permission_engine | allow/deny/ask + 审批记录                             | 不要把安全完全交给 system prompt   |
| sandbox_runtime   | workspace 隔离 + process/network policy             | 不要让 agent 直接运行在宿主环境       |
| state_manager     | checkpoint metadata + artifact refs + action log  | 不要把恢复等同于 messages 恢复      |
| memory            | provenance + scope + version + read/write policy  | 不要先做 “把所有对话全存入向量库”        |
| skill_system      | discover + version + eval + rollback              | 不要把未经验证的成功案例自动升级为共享 Skill |
| observability     | task trace + model/tool spans + cost              | 不要只记录最终文本                 |
| eval              | success rubric + regression cases + safety checks | 不要只使用主观 “感觉更聪明”           |

10.3 一条推荐的生产请求路径

一个更完整的生产请求可以遵循：User/Auth → Task Create → Context Build → Agent Step → Permission Check → Credential Broker → Sandbox/Tool → Action Log → Observation → Checkpoint → Eval → Memory/Skill Candidate → Final Result。关键是每一步都带 task\_id / actor / project / trace\_id，从而把“模型推理”和“真实世界动作”连成一条可审计轨迹。

11. 批判性阅读：两篇文章哪些地方应该打折看？

11.1 第一篇的 Bright Data “97%”数据存在口径问题

【原文】InfoQ 第一篇称 Bright Data 《Data for AI 2026》显示“97% 的企业已经在使用 Agent”。[1] 这一说法需要谨慎：公开可验证资料中，Bright Data 的 97% 更常对应“AI 组织依赖实时 Web 数据基础设施”等口径，而不是可直接等价于“97% 企业已经部署生产 Agent”。因此本报告不把该数字作为 Agent adoption 的可靠基准。

研究方法提示

行业文章里的“企业使用率”“部署率”“试用率”“AI 组织”“受访企业”经常不是同一口径。遇到特别惊人的百分比，优先回到原报告问卷、样本与定义。

11.2 厂商性能数字与案例数据应区分“主张”与“独立测量”

CubeSandbox 的 <60ms 冷启动、<5MB 内存开销来自项目官方基准；InfoQ 文章中的腾讯云并发和 WorkBuddy/CodeBuddy 规模指标来自厂商介绍；ACE Harness 的效率提升来自嘉宾现场分享。[1][2][4] 它们可以证明设计目标和已有实践，但不应直接外推为所有场景都能达到的确定性能。

11.3 “Kubernetes 不适合 Agent”是层次问题，不是二元结论

Agent Runtime 的兴起说明 K8s 不提供足够的 Agent 任务语义，并不意味着 K8s 失去价值。事实上，许多 Agent Runtime 最终仍可能运行在 Kubernetes、VM 或其他基础设施上。真正新增的是 Session、Task、Snapshot、Identity、Trajectory 等更贴近 Agent 的控制面。

11.4 Harness 也可能成为模型能力的“历史包袱”

当模型能力持续提升，过去为了弥补模型弱点写下的提示、工具限制和工作流分支可能逐渐过时。Anthropic 明确提醒 harness 中对模型能力的假设会过时。[10] 所以 Harness 本身也要有 Eval、A/B 测试和删除机制，而不是只增不减。

12. 可期淀的 Know-why / Know-how

12.1 Know-why：为什么这些组件会同时出现？

| 现象          | 底层原因                      | 因此出现的工程组件                                         |
|-------------|---------------------------|---------------------------------------------------|
| 任务越来越长      | Context Window 与真实任务长度不一致 | Compaction / Memory / Handoff / Checkpoint        |
| 模型能行动       | 错误不再停留在文本                 | Permission / Identity / Policy / Approval / Audit |
| 环境会变化       | 恢复对话 ≠ 恢复世界               | Sandbox Snapshot / Artifact State / Action Log    |
| Agent 规模增大  | 实例数量和生命周期更动态              | Agent Runtime / Elasticity / AutoPause            |
| 经验需要复用      | 仅靠 Prompt 不可持续            | Memory / Skill / Eval / Versioning                |
| 多人都使用 Agent | 个人上下文形成碎片                 | Shared Context / Language / Skill /               |

|        |                |                                                  |
|--------|----------------|--------------------------------------------------|
|        |                | Delivery Standard                                |
| 模型持续升级 | 旧 Harness 假设过时 | Harness Eval / Simplification / Adaptive Control |

## 12.2 Know-how：设计任何 Agent 功能时先问 10 个问题

- 26. 它的任务边界是什么？一次调用还是持续任务？
- 27. 需要保存哪些状态？哪些状态属于外部世界？
- 28. 失败后从哪里恢复？哪些动作可以幂等重试？
- 29. Agent 以谁的身份执行？凭证如何最小化、短期化？
- 30. 哪些工具/参数需要策略检查？哪些动作必须人工批准？
- 31. 如何完整追踪 trajectory，而不只记录最终回答？
- 32. 成功标准是什么？能否自动评测？
- 33. 成本是否按 successful task 统计，而非单次请求？
- 34. 成功经验如何进入 Memory / Skill？写入前谁验证？
- 35. 随着模型升级，哪些 Harness 约束应该被删除？

## 12.3 一个更准确的 Agent 方程

**Agent = Model × Context × Tools × Harness × Runtime × Memory × Governance**

这里使用乘法而不是加法，是为了强调“短板效应”：模型再强，如果没有权限边界、状态恢复或可靠工具，生产效果仍可能接近不可用；反过来，基础设施再完整，如果模型能力不足或任务定义错误，也不会自动产生业务价值。

# 13. 给企业/的 90 天落地路线

## 阶段 A：0-30 天 - 建立“可观察的单 Agent”

- 选 1-2 个可衡量、可回滚、边界清晰的任务；不要从全自主跨系统流程开始。
- 统一 task\_id / actor / project / trace\_id；记录每个模型调用、工具调用与最终成功结果。
- 先做工具分发、权限、超时、错误模型和人工审批；把安全从 Prompt 中剥离。
- 建立 20-50 条真实 Eval Case，形成上线前回归。

## 阶段 B：31-60 天 - 建立“可恢复、可治理的 Agent”

- 引入隔离 Sandbox，限制文件、进程和网络；凭证通过代理或短期令牌提供。
- 建立 checkpoint / pause / resume，至少保存任务进度、artifact refs 与 action log。

- 开始按 Cost / Successful Task 统计成本，并分析 retry 与 tool-call 分布。
- 将 3-5 个高频失败模式转成 policy、test 或 skill，而不是只改 system prompt。

### 阶段 C：61-90 天 - 建立“可学习、可共享的组织 Agent”

- 建立带 provenance、scope、actor、version 的长期 Memory；明确读写权限。
- 将反复验证成功的工作方法升级为 Skill，并为 Skill 建立 Eval 与版本回滚。
- 引入 Critic/Judge 或人工 review 到高价值任务，而不是盲目堆叠 Multi-Agent。
- 沉淀统一术语、交付标准和共享上下文，开始治理 Context Fragmentation。

## 14. 最终结论：2026 年 Agent Engineering 正在从“Loop”走向“Operating Environment”

两篇 InfoQ 文章真正有价值的地方，是把 Agent 落地失败的原因从“模型还不够强”拉回系统工程。第一篇从 Runtime 视角说明：Agent 是新的工作负载，需要围绕 Task、State、Sandbox、Identity、Elasticity 和 Observability 重建执行底座。第二篇则说明：Agent 能执行之后，还必须围绕 Memory、Harness、Human-Agent 协作和组织知识治理建立新的生产关系。

因此，Agent 的竞争正在从“谁有更强的模型调用”变成“谁能把概率性的模型能力，嵌入一个确定、可恢复、可观察、可约束、可评测、可学习的系统”。真正可规模化的 Agent，不是一个更复杂的聊天机器人，而是一种新的软件运行与组织协作范式。

#### 最后一句

用系统的确定性，收敛 Agent 的不确定性；再用组织的可学习性，把一次成功变成可复用的生产力。

## 参考资料与证据说明

[1] InfoQ，凌敏，《Agent 狂欢热潮下的冷思考：为什么规模化落地总是陷入僵局？》，2026-07-02。

<https://www.infoq.cn/article/KmDMAvlzBGgwu5A2kf7t>

[2] InfoQ，王玮，《Agent 上岗之后，企业如何治理硅基团队？》，2026-07-03。

<https://www.infoq.cn/news/pNFHkLos3FoDNm8cQsyt>

[3] IDC，《IDC 预测，2030 年 22 亿 AI Agent 将作为“新数字劳动力”席卷全球》，2026-01-12。

<https://www.idc.com/resource-center/blog/>

[idc%E9%A2%84%E6%B5%8B%E4%B8%BA%E6%96%B0%E6%95%B0%E5%AD%97%E5%8A%B3%E5%8A%9B%E5%B8%AD%E5%8D%B7%E5%85%A8%E7%90%83/](https://www.idc.com/resource-center/blog/agent%E9%A2%84%E6%B5%8B%E4%B8%BA%E6%96%B0%E6%95%B0%E5%AD%97%E5%8A%B3%E5%8A%9B%E5%B8%AD%E5%8D%B7%E5%85%A8%E7%90%83/)

[4] TencentCloud / CubeSandbox，项目 README：RustVMM + KVM、快速冷启动、高密度 Sandbox。

<https://github.com/TencentCloud/CubeSandbox>

**[5] TencentCloud / CubeSandbox v0.3.0: CubeCoW Snapshot / Clone / Rollback。**

<https://github.com/TencentCloud/CubeSandbox/discussions/424>

**[6] AWS, Amazon Bedrock AgentCore Overview: Runtime、Memory、Gateway、Identity、Observability、Evaluations 等模块。**

<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html>

**[7] AWS, AgentCore Harness vs. Runtime。**

<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/harness-vs-runtime.html>

**[8] 华为云 AgentArts 基本概念: Agent = Model + Harness。**

[https://support.huaweicloud.com/productdesc-agentarts0/agentarts\\_03\\_0010.html](https://support.huaweicloud.com/productdesc-agentarts0/agentarts_03_0010.html)

**[9] Anthropic Engineering, Harness design for long-running application development, 2026-03-24。**

<https://www.anthropic.com/engineering/harness-design-long-running-apps>

**[10] Anthropic Engineering, Scaling Managed Agents: Decoupling the brain from the hands, 2026-04-08。**

<https://www.anthropic.com/engineering/managed-agents>

证据说明：InfoQ 现场演讲与厂商案例中的性能/效率数字在本报告中均按“原文或官方主张”处理；除非存在独立公开 benchmark，否则不将其外推为通用结果。对文章之外的架构抽象和工程接口，本报告明确标为“推演”。